一、关键介绍
1.1 介绍
1.2 内部 etcd 服务（Stacked etcd topology）(推荐)
1.3 外部 etcd 服务（External etcd topology）
二、具体部署规划
三、准备工作
0、准备4台机器
1、修改主机名及解析(所有节点)
2、关闭一些服务(所有节点)
3、sshd服务优化
4、增大文件打开数量（退出当前会话立即生效）
5、所有节点配置模块自动加载，此步骤不做的话(kubeadm init时会直接失败!)
6、同步集群时间
7、更新基础yum源（所有机器）
8、更新系统软件(排除内核)
9、安装基础常用软件
10、更新内核（docker 对系统内核要求比较高，最好使用4.4+）**主节点操作**
11、所有节点安装IPVS
12、所有机器修改内核参数
四、安装containerd（所有节点都做）
五、部署负载均衡+keepalived
5.1 三台master上部署配置nginx
5.2 三台master部署keepalived
六、安装k8s
6.1 所有机器准备k8s源
6.2 master01上操作
6.3 多个master节点加入集群
（1）上传/统一证书
（2）添加master节点到集群
6.4 worker节点加入集群
6.5 查看所有节点
七、安装网络插件
八、配置自动补全

一、关键介绍

1.1 介绍

部署k8s集群，node节点本身就是多个，没有单点故障，高可用的关键是master相关组件的高可用，负载均衡+keepalived代理多个master对外提供一个vip，所有node节点访问这个vip即可

具体部署方式安装etcd的对接方式不同可以分为两种方案：
1）With stacked control plane nodes（内部 etcd 服务）
2）With an external etcd cluster（外部 etcd 服务）

两种方案各有利弊，在文档 Options for Highly Available topology 中有所描述，需要根据各自需求进行选择。

参考文档：https://kubernetes.io/zh-cn/docs/setup/production-environment/tools/kubeadm/high-availability/

egonlin.com

1.2 内部 etcd 服务（Stacked etcd topology）(推荐)

该类型高可用集群集群，提供数据存储的 etcd 服务在集群（由节点组成，由运行主节点组件的kubeadm管理）内部。

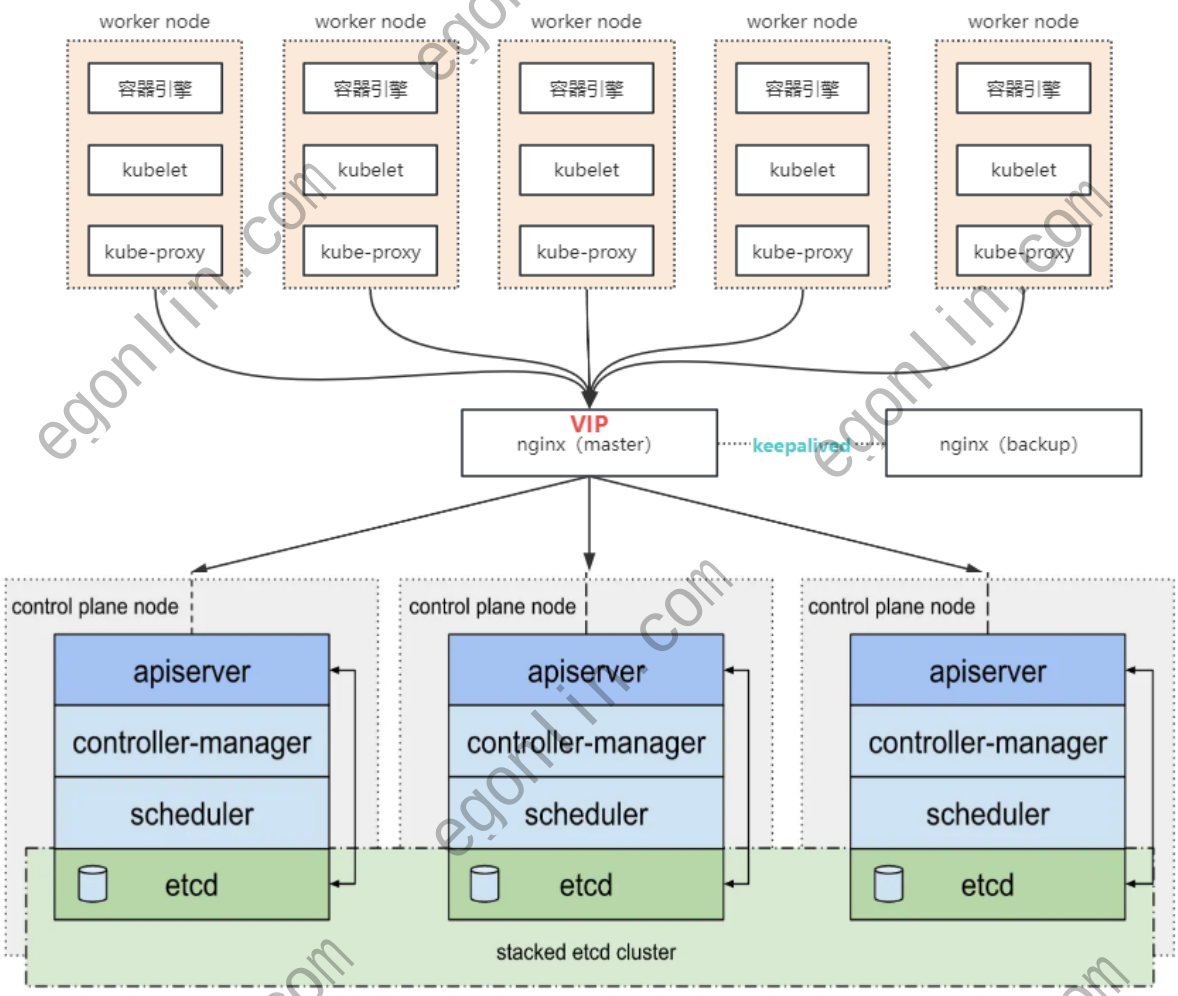
主机节点运行 kube-apiserver、kube-scheduler、kube-controller-manager 组件，本节点上的 kube-controller-manager、kube-scheduler 只与本节点 kube-apiserver 通讯，

每个主机节点会创建本地 etcd 成员，它也是只与本地的 kube-apiserver 通讯。

最后由本节点的kube-apiserver 负责对外暴漏。

优点：
1）节省主机：该集群 etcd 成员与主节点在相同节点，相对于外部 etcd 服务集群，该方案需要的主机数量少；
2）部署简单：集群部署比较简单，复制易于管理；

缺点：
1）鸡蛋都放在一个篮子里了：如果某一个主节点故障，其上的管理组件 kube-apiserver、kube-scheduler、kube-controller-manager 挂掉了，同时该节点上的etcd也一起挂掉了，所以才需要多个主节点



1.3 外部 etcd 服务（External etcd topology）

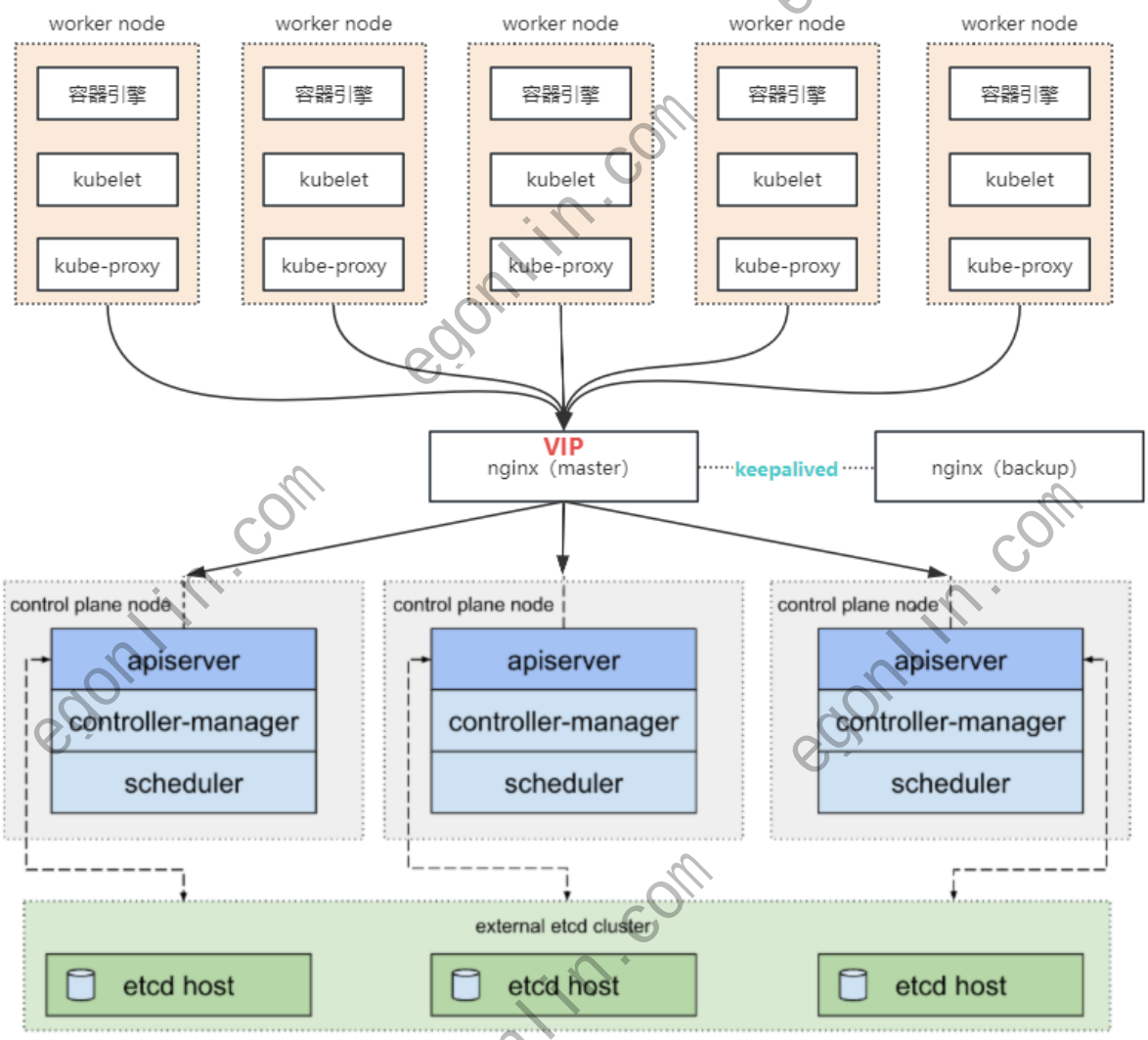
该类型高可用集群集群，提供数据存储的 etcd 服务在集群外部（由节点组成，由运行主节点组件的 kubeadm 管理）。

主机节点运行 kube-apiserver、kube-scheduler、kube-controller-manager 组件，而 kube-apiserver 会暴露为工作节点。但是 etcd 运行在独立主机，每个 etcd 服务与每个主节点上的 kube-apiserver 通讯。

优点：
1）鸡蛋没有放在一个篮子里：etcd放在主节点外的节点上，因此主节点故障或者 etcd 故障不会影响到集群冗余；

缺点：

1）更多主机：至少三台主机运行 etcd 服务，至少三台主机运行主机节点；



https://www.processon.com/diagraming/6136c26d63768906a223f370

二、具体部署规划

主机角色	IP
master01 nginx+keepalived	192.168.71.101
master02 nginx+keepalived	192.168.71.102
master03 nginx+keepalived	192.168.71.103
node01	192.168.71.104

nginx+keepalived 对外提供的VIP地址为：192.168.71.200

三、准备工作

准备工作与你之前非高可用部署时一样

0、准备4台机器

每台机器内存>=2G 核数>=2

1、修改主机名及解析(所有节点)

所有主机修改主机名

```
hostnamectl set-hostname master01
hostnamectl set-hostname master02
hostnamectl set-hostname master03
hostnamectl set-hostname node01
```

所有主机添加解析

```
cat >> /etc/hosts <<EOF
192.168.71.101 master01
192.168.71.102 master02
192.168.71.103 master03
192.168.71.104 node01
192.168.71.200 api-server

EOF
```

2、关闭一些服务(所有节点)

所有主机关闭不必要的服务

```
# 1、关闭selinux
sed -i 's#enforcing#disabled#g' /etc/selinux/config
setenforce 0

# 2、禁用防火墙，网络管理，邮箱
systemctl disable --now firewalld NetworkManager postfix
```

egonlin.com

```
# 3、关闭swap分区
swapoff -a

# 注释swap分区
cp /etc/fstab /etc/fstab_bak
sed -i '/swap/d' /etc/fstab
```

3、sshd服务优化

```
# 1、加速访问（所有节点上）
sed -ri 's@^#UseDNS yes@UseDNS no@g' /etc/ssh/sshd_config
sed -ri 's#^GSSAPIAuthentication yes#GSSAPIAuthentication no#g'
/etc/ssh/sshd_config
grep ^UseDNS /etc/ssh/sshd_config
grep ^GSSAPIAuthentication /etc/ssh/sshd_config
systemctl restart sshd

# 2、密钥登录（主机点做）
# 目的：为了让后续一些远程拷贝操作更方便
ssh-keygen
ssh-copy-id -i root@master01
ssh-copy-id -i root@master02
ssh-copy-id -i root@master03
ssh-copy-id -i root@node01
```

4、增大文件打开数量（退出当前会话立即生效）

```
cat > /etc/security/limits.d/k8s.conf <<'EOF'
* soft nofile 1048576
* hard nofile 1048576
EOF

ulimit -Sn
ulimit -Hn
```

5、所有节点配置模块自动加载，此步骤不做的话(kubeadm init时会直接失败!)

```
modprobe br_netfilter
modprobe ip_conntrack

cat >/etc/rc.sysinit<<EOF
#!/bin/bash
for file in /etc/sysconfig/modules/*.modules ; do
[ -x $file ] && $file
done
EOF

echo "modprobe br_netfilter" >/etc/sysconfig/modules/br_netfilter.modules
echo "modprobe ip_conntrack" >/etc/sysconfig/modules/ip_conntrack.modules
chmod 755 /etc/sysconfig/modules/br_netfilter.modules
chmod 755 /etc/sysconfig/modules/ip_conntrack.modules
```

egonlin.com

```
lsmod | grep br_netfilter
```

6、同步集群时间

我采取的是master01做内网集群的ntp服务端，它与公网ntp服务同步时间

其他节点都跟master01同步时间

```
# ======================》chrony服务端：服务端我们可以自己搭建，也可以直接用公网上的时间服务器，所以是否部署服务端看你自己
# 1、安装
yum -y install chrony

# 2、修改配置文件
mv /etc/chrony.conf /etc/chrony.conf.bak

cat > /etc/chrony.conf << EOF
server ntp1.aliyun.com iburst minpoll 4 maxpoll 10
server ntp2.aliyun.com iburst minpoll 4 maxpoll 10
server ntp3.aliyun.com iburst minpoll 4 maxpoll 10
server ntp4.aliyun.com iburst minpoll 4 maxpoll 10
server ntp5.aliyun.com iburst minpoll 4 maxpoll 10
server ntp6.aliyun.com iburst minpoll 4 maxpoll 10
server ntp7.aliyun.com iburst minpoll 4 maxpoll 10
driftfile /var/lib/chrony/drift
makestep 10 3
rtcsync
allow 0.0.0.0/0
local stratum 10
keyfile /etc/chrony.keys
logdir /var/log/chrony
stratumweight 0.05
noclientlog
logchange 0.5

EOF

# 4、启动chronyd服务
systemctl restart chronyd.service # 最好重启，这样无论原来是否启动都可以重新加载配置
systemctl enable chronyd.service
systemctl status chronyd.service

# ======================》chrony客户端：在需要与外部同步时间的机器上安装，启动后会自动与你指定的服务端同步时间
# 下述步骤一次性粘贴到每个客户端执行即可
# 1、安装chrony
yum -y install chrony
# 2、需改客户端配置文件
/usr/bin/mv /etc/chrony.conf /etc/chrony.conf.bak


cat > /etc/chrony.conf << EOF
# server 服务端的ip地址或可解析的主机名 iburst
server master01 iburst
driftfile /var/lib/chrony/drift
makestep 10 3
rtcsync
```

egonlin.com

```
local stratum 10
keyfile /etc/chrony.key
logdir /var/log/chrony
stratumweight 0.05
noclientlog
logchange 0.5

EOF

# 3、启动chronyd
systemctl restart chronyd.service
systemctl enable chronyd.service
systemctl status chronyd.service

# 4、验证
chronyc sources -v
```

7、更新基础yum源（所有机器）

```
# 1、清理
rm -rf /etc/yum.repos.d/*
yum remove epel-release -y
rm -rf /var/cache/yum/x86_64/6/epel/

# 2、安装阿里的base与epel源
curl -s -o /etc/yum.repos.d/CentOS-Base.repo
https://mirrors.aliyun.com/repo/Centos-7.repo
curl -s -o /etc/yum.repos.d/epel.repo http://mirrors.aliyun.com/repo/epel-7.repo
yum clean all
yum makecache

# 或者用华为的也行
# curl -o /etc/yum.repos.d/CentOS-Base.repo
https://repo.huaweicloud.com/repository/conf/CentOS-7-reg.repo
# yum install -y https://repo.huaweicloud.com/epel/epel-release-latest-
7.noarch.rpm
```

8、更新系统软件(排除内核)

```
yum update -y --exclud=kernel*
```

9、安装基础常用软件

```
yum -y install expect wget jq psmisc vim net-tools telnet yum-utils device-
mapper-persistent-data lvm2 git ntpdate chrony bind-utils rsync unzip git
```

10、更新内核（docker对系统内核要求比较高，最好使用4.4+）主节点操作

egonlin.com

```
wget https://elrepo.org/linux/kernel/el7/x86_64/RPMS/kernel-lt-5.4.274-
1.el7.elrepo.x86_64.rpm
wget https://elrepo.org/linux/kernel/el7/x86_64/RPMS/kernel-lt-devel-5.4.274-
1.el7.elrepo.x86_64.rpm

for i in n1 n2 m1 ; do scp kernel-lt-* $i:/opt; done



补充：如果下载的慢就从网盘里拿吧
链接：https://pan.baidu.com/s/1gVyeBQsJPZjc336E8zGjyQ
提取码：Egon
```

三个节点操作

```
 #安装
yum localinstall -y /opt/kernel-lt*

#调到默认启动
grub2-set-default 0 && grub2-mkconfig -o /etc/grub2.cfg

#查看当前默认启动的内核
grubby --default-kernel

#重启系统
reboot
```

11、所有节点安装IPVS

```
# 1、安装ipvsadm等相关工具
yum -y install ipvsadm ipset sysstat conntrack libseccomp

# 2、配置加载
cat > /etc/sysconfig/modules/ipvs.modules << "EOF"
#!/bin/bash
ipvs_modules="ip_vs ip_vs_lc ip_vs_wlc ip_vs_rr ip_vs_wrr ip_vs_lblc ip_vs_lblcr
ip_vs_dh ip_vs_sh ip_vs_fo ip_vs_nq ip_vs_sed ip_vs_ftp nf_conntrack"
for kernel_module in ${ipvs_modules};
do
    /sbin/modinfo -F filename ${kernel_module} > /dev/null 2>&1
    if [ $? -eq 0 ]; then
        /sbin/modprobe ${kernel_module}
    fi
done

EOF

chmod 755 /etc/sysconfig/modules/ipvs.modules && bash
/etc/sysconfig/modules/ipvs.modules && lsmod | grep ip_vs
```

12、所有机器修改内核参数

```
cat > /etc/sysctl.d/k8s.conf << EOF
net.ipv4.ip_forward = 1
```

egonlin.com

```
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
fs.may_detach_mounts = 1
vm.overcommit_memory=1
vm.panic_on_oom=0
fs.inotify.max_user_watches=89100
fs.file-max=52706963
fs.nr_open=52706963
net.ipv4.tcp_keepalive_time = 600
net.ipv4.tcp.keepaliv.probes = 3
net.ipv4.tcp_keepalive_intvl = 15
net.ipv4.tcp.max_tw_buckets = 36000
net.ipv4.tcp_tw_reuse = 1
net.ipv4.tcp.max_orphans = 327680
net.ipv4.tcp_orphan_retries = 3
net.ipv4.tcp_syncookies = 1
net.ipv4.tcp_max_syn_backlog = 16384
net.ipv4.ip_conntrack_max = 65536
net.ipv4.tcp_max_syn_backlog = 16384
net.ipv4.top_timestamps = 0
net.core.somaxconn = 16384
EOF

# 立即生效
sysctl --system
```

四、安装containerd（所有节点都做）

自Kubernetes1.24以后，K8S就不再原生支持docker了

我们都知道containerd来自于docker，后被docker捐献给了云原生计算基金会(我们安装docker会一并安装上containerd)

1、centos7默认的libseccomp的版本为2.3.1，不满足containerd的需求，需要下载2.4以上的版本即可，我这里部署2.5.1版本。

```
#  1、如果你不升级libseccomp的话，启动容器会报错
**Failed to create pod sandbox: rpc error: code = Unknown desc = failed to
create containerd task: failed to create shim task: OCI runtime create failed:
unable to retrieve OCI runtime error (open
/run/containerd/io.containerd.runtime.v2.task/k8s.io/ed17cbdc31099314dc8fd609d52
b0dfbd6fdf772b78aa26fbc9149ab089c6807/log.json: no such file or directory): runc
did not terminate successfully: exit status 127: unknown**

# 2、升级
rpm -e libseccomp-2.3.1-4.el7.x86_64 --nodeps
# wget http://rpmfind.net/linux/centos/8-
stream/BaseOS/x86_64/os/Packages/libseccomp-2.5.1-1.el8.x86_64.rpm
wget https://mirrors.aliyun.com/centos/8/BaseOS/x86_64/os/Packages/libseccomp-
2.5.1-1.el8.x86_64.rpm
rpm -ivh libseccomp-2.5.1-1.el8.x86_64.rpm  # 官网已经gg了，不更新了，请用阿里云

rpm -qa | grep libseccomp
```

安装方式一：（基于阿里云的源）推荐用这种方式，安装的是

egonlin.com

```
# 1、卸载之前的
yum remove docker docker-ce containerd docker-common docker-selinux docker-engine
-y

# 2、准备repo
cd /etc/yum.repos.d/
wget http://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo

# 3、安装
yum install containerd* -y
```

了解--->安装方式二：或者去官网找最新的包（新版1.7.18包内缺少runc容器运行时，需要单独安，旧版1.6.4是包含runc的，不需要额外安装）

```
# 1、下载containerd：https://github.com/containerd/containerd/releases/
wget
https://github.com/containerd/containerd/releases/download/v1.7.18/containerd-
1.7.18-linux-amd64.tar.gz
# 国内下载地址：https://gitee.com/egonlin/containerd-1.7.18

# 注意新版1.7.18包内缺少runc容器运行时，需要单独安装
# 而wget https://github.com/containerd/containerd/releases/download/v1.6.4/cri-
containerd-cni-1.6.4-linux-amd64.tar.gz该版本中包含了 containerd以及cri runc等相关工
具包


# 2、解压即可
tar zxvf containerd-1.7.18-linux-amd64.tar.gz -C /usr  # 命令都会解压到/usr/bin下，
可以直接用都不用处理PATH变量

# 3、需要自己添加系统服务
cat > /usr/lib/systemd/system/containerd.service << "EOF"
[Unit]
Description=containerd container runtime
Documentation=https://containerd.io
After=network.target local-fs.target

[Service]
ExecStartPre=-/sbin/modprobe overlay
ExecStart=/usr/bin/containerd

Type=notify
Delegate=yes
KillMode=process
Restart=always
RestartSec=5
# Having non-zero Limit*s causes performance problems due to accounting overhead
# in the kernel. We recommend using cgroups to do container-local accounting.
LimitNPROC=infinity
LimitCORE=infinity
LimitNOFILE=infinity
# Comment TasksMax if your systemd version does not supports it.
# Only systemd 226 and above support this version.
TasksMax=infinity
```

```
OOMScoreAdjust=-999

[Install]
WantedBy=multi-user.target

EOF

# 3、
systemctl daemon-reload

# 4、注意1.7.18包内缺少runc容器运行时，需要单独安装，详见附录
```

配置

```
# 1、配置
mkdir -pv /etc/containerd
containerd config default > /etc/containerd/config.toml # 为containerd生成配置文件

# 2、替换默认pause镜像地址: 这一步非常非常非常非常重要
#  这一步非常非常非常非常重要，国内的镜像地址可能导致下载失败，最终kubeadm安装失
败！！！！！！！！！！！！！
#  这一步非常非常非常非常重要，国内的镜像地址可能导致下载失败，最终kubeadm安装失
败！！！！！！！！！！！！！
#  这一步非常非常非常非常重要，国内的镜像地址可能导致下载失败，最终kubeadm安装失
败！！！！！！！！！！！！！
grep sandbox_image /etc/containerd/config.toml
sed -i 's/registry.k8s.io/registry.cn-hangzhou.aliyuncs.com\/google_containers/'
/etc/containerd/config.toml
grep sandbox_image /etc/containerd/config.toml
# 请务必确认新地址是可用的：sandbox_image = "registry.cn-
hangzhou.aliyuncs.com/google_containers/pause:3.6"

# 3、配置systemd作为容器的cgroup driver
grep SystemdCgroup /etc/containerd/config.toml
sed -i 's/SystemdCgroup \= false/SystemdCgroup \= true/'
/etc/containerd/config.toml
grep SystemdCgroup /etc/containerd/config.toml


# 4、配置加速器（必须配置，否则后续安装cni网络插件时无法从docker.io里下载镜像）
#参考：
https://github.com/containerd/containerd/blob/main/docs/cri/config.md#registry-
configuration
#添加 config_path = "/etc/containerd/certs.d"
sed -i 's/config_path\ =.*/config_path = \"\/etc\/containerd\/certs.d\"/g'
/etc/containerd/config.toml
mkdir /etc/containerd/certs.d/docker.io -p

# docker hub镜像加速
mkdir -p /etc/containerd/certs.d/docker.io
cat > /etc/containerd/certs.d/docker.io/hosts.toml << EOF
server = "https://docker.io"
[host."https://dockerproxy.com"]
  capabilities = ["pull", "resolve"]
```

```
[host."https://docker.m.daocloud.io"]
  capabilities = ["pull", "resolve"]


[host."https://registry.docker-cn.com"]
  capabilities = ["pull", "resolve"]

[host."http://hub-mirror.c.163.com"]
  capabilities = ["pull", "resolve"]

EOF

# 5、配置containerd开机自启动

# 5.1 启动containerd服务并配置开机自启动
systemctl daemon-reload && systemctl restart containerd
systemctl enable --now containerd

# 5.2 查看containerd状态
systemctl status containerd

# 5.3 查看containerd的版本
ctr version
```

五、部署负载均衡+keepalived

部署负载均衡+keepalived对外提供vip：192.168.71.200

5.1 三台master上部署配置nginx

```
# 1、添加repo源
cat > /etc/yum.repos.d/nginx.repo << "EOF"
[nginx-stable]
name=nginx stable repo
baseurl=http://nginx.org/packages/centos/$releasever/$basearch/
gpgcheck=1
enabled=1
gpgkey=https://nginx.org/keys/nginx_signing.key
module_hotfixes=true

[nginx-mainline]
name=nginx mainline repo
baseurl=http://nginx.org/packages/mainline/centos/$releasever/$basearch/
gpgcheck=1
enabled=0
gpgkey=https://nginx.org/keys/nginx_signing.key
module_hotfixes=true

EOF

# 2、安装
```

egonlin.com

```
yum install nginx -y


# 3、配置
cat > /etc/nginx/nginx.conf <<'EOF'
user nginx nginx;
worker_processes auto;

events {
    worker_connections  20240;
    use epoll;
}

error_log /var/log/nginx_error.log info;

stream {
    upstream kube-servers {
        hash $remote_addr consistent;
        server master01:6443 weight=5 max_fails=1 fail_timeout=3s;
        server master02:6443 weight=5 max_fails=1 fail_timeout=3s;
        server master03:6443 weight=5 max_fails=1 fail_timeout=3s;
    }

    server {
        listen 8443 reuseport;  # 监听8443端口
        proxy_connect_timeout 3s;
        proxy_timeout 3000s;
        proxy_pass kube-servers;
    }
}
EOF

# 4、启动
systemctl restart nginx
systemctl enable nginx
systemctl status nginx
```

5.2 三台master部署keepalived

1、安装

```
yum  -y install  keepalived
```

2、修改keepalive的配置文件(根据实际环境，interface eth0可能需要修改为interface ens33)

```
# 编写配置文件，各个master节点需要修改router_id和mcast_src_ip的值即可。

# ==================================> master01
cat > /etc/keepalived/keepalived.conf <<EOF
! Configuration File for keepalived
global_defs {
   router_id 192.168.71.101
}
vrrp_script chk_nginx {
    script "/etc/keepalived/check_port.sh 8443"
```

egonlin.com

```
    interval 2
    weight -20
}
vrrp_instance VI_1 {
    state BACKUP
    interface ens36
    virtual_router_id 100
    priority 100
    advert_int 1
    mcast_src_ip 192.168.71.101
    # nopreempt # 这行注释掉，否则即使一个具有更高优先级的备份节点出现，当前的 MASTER 也不会被抢占，直至 MASTER 失效。
    authentication {
        auth_type PASS
        auth_pass 11111111
    }
    track_script {
         chk_nginx
    }
    virtual_ipaddress {
        192.168.71.200
    }
}
EOF


# ===================================> master02
cat > /etc/keepalived/keepalived.conf <<EOF
! Configuration File for keepalived
global_defs {
   router_id 192.168.71.102
}
vrrp_script chk_nginx {
    script "/etc/keepalived/check_port.sh 8443"
    interval 2
    weight -20
}
vrrp_instance VI_1 {
    state BACKUP
    interface ens36
    virtual_router_id 100
    priority 100
    advert_int 1
    mcast_src_ip 192.168.71.102
    # nopreempt # 这行注释掉，否则即使一个具有更高优先级的备份节点出现，当前的 MASTER 也不会被抢占，直至 MASTER 失效。
    authentication {
        auth_type PASS
        auth_pass 11111111
    }
    track_script {
         chk_nginx
    }
    virtual_ipaddress {
        192.168.71.200
    }
}
EOF
```

```
# ===================================> master03
cat > /etc/keepalived/keepalived.conf <<EOF
! Configuration File for keepalived
global_defs {
   router_id 192.168.71.103
}
vrrp_script chk_nginx {
    script "/etc/keepalived/check_port.sh 8443"
    interval 2
    weight -20
}
vrrp_instance VI_1 {
    state BACKUP
    interface ens36
    virtual_router_id 100
    priority 100
    advert_int 1
    mcast_src_ip 192.168.71.103
    # nopreempt # 这行注释掉，否则即使一个具有更高优先级的备份节点出现，当前的 MASTER 也不会被抢占，直至 MASTER 失效。
    authentication {
        auth_type PASS
        auth_pass 11111111
    }
    track_script {
         chk_nginx
    }
    virtual_ipaddress {
        192.168.71.200
    }
}
EOF
```

3、所有master节点上创建健康检查脚本

```
cat > /etc/keepalived/check_port.sh << 'EOF'
# 设置环境变量，确保所有必要的命令路径正确
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

CHK_PORT=$1
if [ -n "$CHK_PORT" ];then
    PORT_PROCESS=$(/usr/sbin/ss -lt | grep ":$CHK_PORT" | wc -l)
    if [ $PORT_PROCESS -eq 0 ];then
        echo "Port $CHK_PORT Is Not Used,End."
        exit 1
    fi
else
    echo "Check Port Cant Be Empty!"
fi
EOF

chmod +x /etc/keepalived/check_port.sh
```

4、启动

egonlin.com

```
systemctl restart keepalived
systemctl enable keepalived
systemctl status keepalived
```

5、在某个master节点上可以看到vip

```
[root@master01 ~]# ip addr show dev ens36
2: ens36: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP
group default qlen 1000
    link/ether 00:0c:29:30:fb:e2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.71.101/24 brd 192.168.71.255 scope global ens36
       valid_lft forever preferred_lft forever
    inet 192.168.71.200/32 scope global ens36 # ---------------------------》
vip在这里
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe30:fbe2/64 scope link
       valid_lft forever preferred_lft forever

[root@master02 ~]# ip addr show dev ens36
2: ens36: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP
group default qlen 1000
    link/ether 00:0c:29:6a:1d:22 brd ff:ff:ff:ff:ff:ff
    inet 192.168.71.102/24 brd 192.168.71.255 scope global ens36
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe6a:1d22/64 scope link
       valid_lft forever preferred_lft forever


[root@master03 ~]# ip addr show dev ens36
2: ens36: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP
group default qlen 1000
    link/ether 00:0c:29:d1:ee:2d brd ff:ff:ff:ff:ff:ff
    inet 192.168.71.103/24 brd 192.168.71.255 scope global ens36
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fed1:ee2d/64 scope link
       valid_lft forever preferred_lft forever
```

6、去到master01上停掉nginx，8443端口就没了

```
[root@master01 ~]# systemctl stop nginx

会发现vip漂移走了，注意因为你的检测脚本/etc/keepalived/check_port.sh检测端口失效exit非0
后，当前master的权重会-20，此时想其他节点能够抢走vip，你必须注释掉 nopreempt
```

7、动态查看keepalived日志

```
journalctl -u keepalived -f
```

六、安装k8s

egonlin.com

6.1 所有机器准备k8s源

```
cat > /etc/yum.repos.d/kubernetes.repo << "EOF"
[kubernetes]
name=Kubernetes
baseurl=https://mirrors.aliyun.com/kubernetes-new/core/stable/v1.30/rpm/
enabled=1
gpgcheck=1
gpgkey=https://mirrors.aliyun.com/kubernetes-
new/core/stable/v1.30/rpm/repodata/repomd.xml.key
EOF

# 参考：https://developer.aliyun.com/mirror/kubernetes/
```

所有机器都要安装

```
yum install -y kubelet-1.30* kubeadm-1.30* kubectl-1.30*
systemctl enable kubelet && systemctl start kubelet && systemctl status kubelet
```

6.2 master01上操作

```
初始化master节点(仅在master01节点上执行)：
# 可以kubeadm config images list查看
[root@maste01 ~]# kubeadm config images list
registry.k8s.io/kube-apiserver:v1.30.0
registry.k8s.io/kube-controller-manager:v1.30.0
registry.k8s.io/kube-scheduler:v1.30.0
registry.k8s.io/kube-proxy:v1.30.0
registry.k8s.io/coredns/coredns:v1.11.1
registry.k8s.io/pause:3.9
registry.k8s.io/etcd:3.5.12-0
```

先生成配置文件，编辑修改后，再部署（推荐此方式，因为高级配置只能通过配置文件指定，例如配置使用ipvs模式直接用kubeadm init则无法指定）

```
kubeadm config print init-defaults > kubeadm.yaml # 先生成配置文件，内容及修改如下

apiVersion: kubeadm.k8s.io/v1beta3
bootstrapTokens:
- groups:
  - system:bootstrappers:kubeadm:default-node-token
  token: abcdef.0123456789abcdef
  ttl: 24h0m0s
  usages:
  - signing
  - authentication
kind: InitConfiguration
localAPIEndpoint:
  advertiseAddress: 0.0.0.0 # 统一监听在0.0.0.0即可
```

egonlin.com

```
  bindPort: 6443
nodeRegistration:
  criSocket: unix:///var/run/containerd/containerd.sock #指定containerd容器运行时
  imagePullPolicy: IfNotPresent
  name: master01 # 你当前的主机名
  taints: null
---
apiServer:
  timeoutForControlPlane: 4m0s
apiVersion: kubeadm.k8s.io/v1beta3
certificatesDir: /etc/kubernetes/pki
clusterName: kubernetes
controllerManager: {}
dns: {}
etcd:
  local:
    dataDir: /var/lib/etcd  # 内部etcd服务就直接指定本地文件夹就行
imageRepository: registry.cn-hangzhou.aliyuncs.com/google_containers # 换成阿里云镜像仓库地址
kind: ClusterConfiguration
kubernetesVersion: 1.30.0  # 指定k8s版本
controlPlaneEndpoint: "api-server:8443"  # 指定你的vip地址192.168.71.200与负载均可暴漏的端口，建议用主机名
networking:
  dnsDomain: cluster.local
  serviceSubnet: 10.96.0.0/12 # 指定Service网段
  podSubnet: 10.244.0.0/16 # 增加一行，指定pod网段
scheduler: {}

#在文件最后，插入以下内容，（复制时，要带着---）：
---
apiVersion: kubeproxy.config.k8s.io/v1alpha1
kind: KubeProxyConfiguration
mode: ipvs # 表示kube-proxy代理模式是ipvs，如果不指定ipvs，会默认使用iptables，但是iptables效率低，所以我们生产环境建议开启ipvs，阿里云和华为云托管的K8s，也提供ipvs模式
---
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
cgroupDriver: systemd
```

部署

```
[root@master01 ~]# kubeadm init --config=kubeadm.yaml --ignore-preflight-errors=SystemVerification --ignore-preflight-errors=Swap
```

部署过程中监听报错日志

```
[root@master01 ~]# journalctl -u kubelet -f

如果安装报错，请解决错误，
然后清理：kubeadm reset -f
然后再重新安装
```

egonlin.com

输出结果（如果你各步骤都正确的话，这个过程会非常快，如果你原地阻塞了，一般都是有问题）

```
[root@master01 ~]# kubeadm init --config=kubeadm.yaml --ignore-preflight-
errors=SystemVerification --ignore-preflight-errors=Swap
[init] Using Kubernetes version: v1.30.0
[preflight] Running pre-flight checks
[preflight] Pulling images required for setting up a Kubernetes cluster
[preflight] This might take a minute or two, depending on the speed of your
internet connection
[preflight] You can also perform this action in beforehand using 'kubeadm config
images pull'
W0619 22:04:15.474723  137005 checks.go:844] detected that the sandbox image
"registry.cn-hangzhou.aliyuncs.com/google_containers/pause:3.6" of the container
runtime is inconsistent with that used by kubeadm.It is recommended to use
"registry.cn-hangzhou.aliyuncs.com/google_containers/pause:3.9" as the CRI
sandbox image.
[certs] Using certificateDir folder "/etc/kubernetes/pki"
[certs] Generating "ca" certificate and key
[certs] Generating "apiserver" certificate and key
[certs] apiserver serving cert is signed for DNS names [api-server kubernetes
kubernetes.default kubernetes.default.svc kubernetes.default.svc.cluster.local
master01] and IPs [10.96.0.1 192.168.71.101]
[certs] Generating "apiserver-kubelet-client" certificate and key
[certs] Generating "front-proxy-ca" certificate and key
[certs] Generating "front-proxy-client" certificate and key
[certs] Generating "etcd/ca" certificate and key
[certs] Generating "etcd/server" certificate and key
[certs] etcd/server serving cert is signed for DNS names [localhost master01] and
IPs [192.168.71.101 127.0.0.1 ::1]
[certs] Generating "etcd/peer" certificate and key
[certs] etcd/peer serving cert is signed for DNS names [localhost master01] and
IPs [192.168.71.101 127.0.0.1 ::1]
[certs] Generating "etcd/healthcheck-client" certificate and key
[certs] Generating "apiserver-etcd-client" certificate and key
[certs] Generating "sa" key and public key
[kubeconfig] Using kubeconfig folder "/etc/kubernetes"
W0619 22:04:17.687580  137005 endpoint.go:57] [endpoint] WARNING: port specified
in controlPlaneEndpoint overrides bindPort in the controlplane address
[kubeconfig] Writing "admin.conf" kubeconfig file
W0619 22:04:17.894478  137005 endpoint.go:57] [endpoint] WARNING: port specified
in controlPlaneEndpoint overrides bindPort in the controlplane address
[kubeconfig] Writing "super-admin.conf" kubeconfig file
W0619 22:04:18.102146  137005 endpoint.go:57] [endpoint] WARNING: port specified
in controlPlaneEndpoint overrides bindPort in the controlplane address
[kubeconfig] Writing "kubelet.conf" kubeconfig file
W0619 22:04:18.220674  137005 endpoint.go:57] [endpoint] WARNING: port specified
in controlPlaneEndpoint overrides bindPort in the controlplane address
[kubeconfig] Writing "controller-manager.conf" kubeconfig file
W0619 22:04:18.290253  137005 endpoint.go:57] [endpoint] WARNING: port specified
in controlPlaneEndpoint overrides bindPort in the controlplane address
[kubeconfig] Writing "scheduler.conf" kubeconfig file
[etcd] Creating static Pod manifest for local etcd in
"/etc/kubernetes/manifests"
[control-plane] Using manifest folder "/etc/kubernetes/manifests"
[control-plane] Creating static Pod manifest for "kube-apiserver"
[control-plane] Creating static Pod manifest for "kube-controller-manager"
[control-plane] Creating static Pod manifest for "kube-scheduler"
```

egonlin.com

```
[kubelet-start] Writing kubelet environment file with flags to file
"/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Writing kubelet configuration to file
"/var/lib/kubelet/config.yaml"
[kubelet-start] Starting the kubelet
[wait-control-plane] Waiting for the kubelet to boot up the control plane as
static Pods from directory "/etc/kubernetes/manifests"
[kubelet-check] Waiting for a healthy kubelet. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 501.003502ms
[api-check] Waiting for a healthy API server. This can take up to 4m0s
[api-check] The API server is healthy after 4.503485399s
[upload-config] Storing the configuration used in ConfigMap "kubeadm-config" in
the "kube-system" Namespace
[kubelet] Creating a ConfigMap "kubelet-config" in namespace kube-system with the
configuration for the kubelets in the cluster
[upload-certs] Skipping phase. Please see --upload-certs
[mark-control-plane] Marking the node master01 as control-plane by adding the
labels: [node-role.kubernetes.io/control-plane node.kubernetes.io/exclude-from-
external-load-balancers]
[mark-control-plane] Marking the node master01 as control-plane by adding the
taints [node-role.kubernetes.io/control-plane:NoSchedule]
[bootstrap-token] Using token: abcdef.0123456789abcdef
[bootstrap-token] Configuring bootstrap tokens, cluster-info ConfigMap, RBAC
Roles
[bootstrap-token] Configured RBAC rules to allow Node Bootstrap tokens to get
nodes
[bootstrap-token] Configured RBAC rules to allow Node Bootstrap tokens to post
CSRs in order for nodes to get long term certificate credentials
[bootstrap-token] Configured RBAC rules to allow the csrapprover controller
automatically approve CSRs from a Node Bootstrap Token
[bootstrap-token] Configured RBAC rules to allow certificate rotation for all
node client certificates in the cluster
[bootstrap-token] Creating the "cluster-info" ConfigMap in the "kube-public"
namespace
[kubelet-finalize] Updating "/etc/kubernetes/kubelet.conf" to point to a
rotatable kubelet client certificate and key
[addons] Applied essential addon: CoreDNS
W0619 22:04:25.913392  137005 endpoint.go:57] [endpoint] WARNING: port specified
in controlPlaneEndpoint overrides bindPort in the controlplane address
[addons] Applied essential addon: kube-proxy

Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

  export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
  https://kubernetes.io/docs/concepts/cluster-administration/addons/
```

```
You can now join any number of control-plane nodes by copying certificate
authorities
and service account keys on each node and then running the following as root:

  kubeadm join api-server:8443 --token abcdef.0123456789abcdef \
    --discovery-token-ca-cert-hash
sha256:4c189f138d96e684a7f74ef0c9cabcf3eb7bf5ab7559e9f5988ff7ba9d9011cb \
    --control-plane

Then you can join any number of worker nodes by running the following on each as
root:

kubeadm join api-server:8443 --token abcdef.0123456789abcdef \
    --discovery-token-ca-cert-hash
sha256:4c189f138d96e684a7f74ef0c9cabcf3eb7bf5ab7559e9f5988ff7ba9d9011cb
```

根据上述输出结果执行命令

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

查看cm资源

```
kubectl -n kube-system get cm kubeadm-config -o yaml
```

6.3 多个master节点加入集群

（1）上传/统一证书

将集群控制平面所需的证书上传到集群中，并存储在一个叫做 `kubeadm-certs` 的 Secret 中

```
[root@master01 ~]# kubeadm init phase upload-certs --upload-certs # 你也可以在安装的时候kubeadm init的默认加上选项--upload-certs来上传证书，那这一步就不需要了

输出结果
...
[upload-certs] Using certificate key:
75fb3ae07478f5777b187e45cd5e85cdf9f3bd8dbeeaf5919416e9022b394dea  # 得到该值，后续添加master需要用到它
```

上述指令做了3件事

1. **上传证书到集群**：将已经生成的控制平面证书上传到 Kubernetes 集群，使得这些证书可以在多个 Master 节点之间共享。
2. **生成证书密钥**：生成一个唯一的密钥用于加密和保护这些证书。
3. **输出证书密钥**：在终端输出这个证书密钥，后续添加新的 Master 节点时需要使用。

上述指令目的是：

- 方便其他 Master 节点在加入集群时能够共享这些证书。这个步骤极大简化了多 Master 节点的配置过程，确保所有 Master 节点使用相同的证书。

egonlin.com

是否一定要做这一步？

- 不一定。如果你没有这一步骤，那么每一个新加入的 Master 节点会自己生成一份新的证书，这样可能导致集群内的证书不一致，增加管理复杂度。使用 `kubeadm init phase upload-certs --upload-certs` 可以确保所有控制平面节点使用相同的证书，简化管理并且提高安全性和一致性。因此，尽管这一步不是绝对必要的，但强烈推荐使用它。

（2）添加master节点到集群

所有master节点加入集群，需要用到上面的token信息，需要使用"--certificate-key"选项指定。（你的环境要替换！）

master02节点上

```
kubeadm join api-server:8443 --token abcdef.0123456789abcdef \
    --discovery-token-ca-cert-hash
sha256:4c189f138d96e684a7f74ef0c9cabcf3eb7bf5ab7559e9f5988ff7ba9d9011cb \
    --control-plane \
    --certificate-key
75fb3ae07478f5777b187e45cd5e85cdf9f3bd8dbeeaf5919416e9022b394dea


成功后按照提示执行
mkdir -p $HOME/.kube
    sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
    sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

master03节点上

```
kubeadm join api-server:8443 --token abcdef.0123456789abcdef \
    --discovery-token-ca-cert-hash
sha256:4c189f138d96e684a7f74ef0c9cabcf3eb7bf5ab7559e9f5988ff7ba9d9011cb \
    --control-plane \
    --certificate-key
75fb3ae07478f5777b187e45cd5e85cdf9f3bd8dbeeaf5919416e9022b394dea


成功后按照提示执行
mkdir -p $HOME/.kube
    sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
    sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

6.4 worker节点加入集群

在node01上执行

```
kubeadm join api-server:8443 --token abcdef.0123456789abcdef \
    --discovery-token-ca-cert-hash
sha256:4c189f138d96e684a7f74ef0c9cabcf3eb7bf5ab7559e9f5988ff7ba9d9011cb
```

显示信息->代表成功

egonlin.com

```
[root@node01 /etc/yum.repos.d]# kubeadm join api-server:8443 --token
abcdef.0123456789abcdef \
> --discovery-token-ca-cert-hash
sha256:4c189f138d96e684a7f74ef0c9cabcf3eb7bf5ab7559e9f5988ff7ba9d9011cb
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system
get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file
"/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file
"/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 501.060793ms
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.
```

6.5 查看所有节点

在master节点上执行查看node的指令：最开始时NotReady状态正常，因为网络组件没有部署ok

```
[root@master01 ~]# kubectl  get nodes
NAME        STATUS     ROLES           AGE     VERSION
master01    NotReady   control-plane   14m     v1.30.2
master02    NotReady   control-plane   2m44s   v1.30.2
master03    NotReady   control-plane   98s     v1.30.2
node01      NotReady   <none>          10s     v1.30.2



[root@master01 ~]# kubectl -n kube-system get pods
NAME                                 READY   STATUS    RESTARTS   AGE
coredns-7c445c467-qsdxm              0/1     Pending   0          15m   # 因为网络组件
尚未安装
coredns-7c445c467-sjtvp              0/1     Pending   0          15m
etcd-master01                        1/1     Running   1          15m
etcd-master02                        1/1     Running   0          3m47s
etcd-master03                        1/1     Running   0          2m40s
kube-apiserver-master01              1/1     Running   1          15m
kube-apiserver-master02              1/1     Running   0          3m47s
kube-apiserver-master03              1/1     Running   0          2m40s
kube-controller-manager-master01     1/1     Running   1          15m
kube-controller-manager-master02     1/1     Running   0          3m47s
kube-controller-manager-master03     1/1     Running   0          2m40s
kube-proxy-9szcf                     1/1     Running   0          3m49s
kube-proxy-gm9tc                     1/1     Running   0          75s
kube-proxy-rlmb6                     1/1     Running   0          2m43s
```

```
kube-proxy-vf5rh                    1/1     Running   0          15m
kube-scheduler-master01             1/1     Running   1          15m
kube-scheduler-master02             1/1     Running   0          3m47s
kube-scheduler-master03             1/1     Running   0          2m40s
[root@master01 ~]#
```

七、安装网络插件

下载网络插件

```
 wget https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml

[root@master01 flannel]# vim kube-flannel.yml

apiVersion: v1
data:
  ...
  net-conf.json: |
    {
      "Network": "10.244.0.0/16", # 与--pod-network-cidr保持一致
      "Backend": {
        "Type": "vxlan"
      }
    }
```

在master01上部署

```
kubectl apply -f kube-flannel.yml
kubectl -n kube-flannel get pods
kubectl -n kube-flannel get pods -w  # 过程中可能会报镜像下载失败，因为我们在第四章节配置的镜像加速慢的问题，等一会就可以了
```

如下（你可以手动delete触发重启来重新下载镜像，需要去指定节点systemctl restart kubelet会快速触发删除pod的操作，否则你delete pod之后长时间处于Terminating状态）

```
[root@master01 ~]# kubectl  -n kube-flannel get pods -o wide -w
NAME                    READY   STATUS                  RESTARTS   AGE     IP
          NODE        NOMINATED NODE   READINESS GATES
kube-flannel-ds-42f9z   0/1     Init:ErrImagePull       0          3m46s
192.168.71.104   node01     <none>           <none>
kube-flannel-ds-hsc2p   1/1     Running                 0          3m46s
192.168.71.103   master03   <none>           <none>
kube-flannel-ds-m5cdb   0/1     Init:1/2                0          3m46s
192.168.71.102   master02   <none>           <none>
kube-flannel-ds-wc78c   0/1     Init:ImagePullBackOff   0          3m46s
192.168.71.101   master01   <none>           <none>
```

egonlin.com

```
kube-flannel-ds-wc78c   0/1     Init:ErrImagePull       0          3m53s
192.168.71.101   master01   <none>           <none>
kube-flannel-ds-wc78c   0/1     Init:1/2                0          3m54s
192.168.71.101   master01   <none>           <none>
kube-flannel-ds-wc78c   0/1     PodInitializing         0          3m55s
192.168.71.101   master01   <none>           <none>
kube-flannel-ds-wc78c   1/1     Running                 0          3m56s
192.168.71.101   master01   <none>           <none>
```

最后都趋于正常

```
[root@master01 ~]# kubectl  -n kube-flannel get pods -o wide -w
NAME                    READY   STATUS    RESTARTS   AGE     IP
NODE       NOMINATED NODE   READINESS GATES
kube-flannel-ds-4td46   1/1     Running   0          5m33s   192.168.71.104
node01     <none>           <none>
kube-flannel-ds-hsc2p   1/1     Running   0          12m     192.168.71.103
master03   <none>           <none>
kube-flannel-ds-hzk8p   1/1     Running   0          5m23s   192.168.71.102
master02   <none>           <none>
kube-flannel-ds-wc78c   1/1     Running   0          12m     192.168.71.101
master01   <none>           <none>
```

你实在觉得慢，就用我们之前交给你的方法，把国外镜像搞成国内的

待上述pod完成后查看

```
[root@master01 ~]# kubectl get nodes # 全部ready
NAME       STATUS   ROLES           AGE   VERSION
master01   Ready    control-plane   37m   v1.30.2
master02   Ready    control-plane   26m   v1.30.2
master03   Ready    control-plane   25m   v1.30.2
node01     Ready    <none>          23m   v1.30.2
[root@master01 ~]# kubectl -n kube-system get pods # # 两个coredns的pod也都ready
NAME                                READY   STATUS    RESTARTS   AGE
coredns-7c445c467-qsdxm             1/1     Running   0          37m
coredns-7c445c467-sjtvp             1/1     Running   0          37m
etcd-master01                       1/1     Running   1          37m
etcd-master02                       1/1     Running   0          26m
etcd-master03                       1/1     Running   0          25m
kube-apiserver-master01             1/1     Running   1          37m
kube-apiserver-master02             1/1     Running   0          26m
kube-apiserver-master03             1/1     Running   0          25m
kube-controller-manager-master01    1/1     Running   1          37m
kube-controller-manager-master02    1/1     Running   0          26m
kube-controller-manager-master03    1/1     Running   0          25m
kube-proxy-9szcf                    1/1     Running   0          26m
kube-proxy-gm9tc                    1/1     Running   0          23m
kube-proxy-rlmb6                    1/1     Running   0          25m
kube-proxy-vf5rh                    1/1     Running   0          37m
kube-scheduler-master01             1/1     Running   1          37m
```

```
kube-scheduler-master02            1/1     Running   0          26m
kube-scheduler-master03            1/1     Running   0          25m
```

部署测试yaml

```
kind: Deployment
apiVersion: apps/v1
metadata:
  name: emptydir
spec:
  selector:
    matchLabels:
      app: emptydir
  template:
    metadata:
      labels:
        app: emptydir
    spec:
      containers:
        - name: nginx
          image: nginx:1.18
```

部署

```
[root@master01 ~]# kubectl apply -f test.yaml

[root@master01 ~]# kubectl  get pods -o wide
NAME                        READY   STATUS    RESTARTS   AGE   IP           NODE
  NOMINATED NODE   READINESS GATES
emptydir-fcb799f4d-44p4q   1/1     Running   0          87s   10.244.3.3   node01
  <none>           <none>
```

访问测试

```
[root@master01 ~]# curl -I 10.244.3.3
HTTP/1.1 200 OK
Server: nginx/1.18.0
Date: Wed, 19 Jun 2024 14:52:20 GMT
Content-Type: text/html
Content-Length: 612
Last-Modified: Tue, 21 Apr 2020 14:09:01 GMT
Connection: keep-alive
ETag: "5e9efe7d-264"
Accept-Ranges: bytes
```

八、配置自动补全

egonlin.com

```
yum -y install bash-completion

kubectl completion bash > ~/.kube/completion.bash.inc
echo "source '$HOME/.kube/completion.bash.inc'" >> $HOME/.bash_profile
source $HOME/.bash_profile
```

egonlin.com